



Software Engineering

Introduction



- Software Engineering is the systematic application of engineering principles to the design, development, testing, deployment, and maintenance of software systems.
- In simple terms:
 - It's about building high-quality, reliable, and maintainable software in a structured, disciplined way — not just coding!
- **Goals of Software Engineering**
 - Build software that works correctly and efficiently
 - Ensure it's maintainable, scalable, and secure
 - Deliver software on time and within budget
 - Satisfy user needs and business requirements.

Software

package {
→ program → Executable | source code
→ dependencies
↳ config files, resources, libraries, frameworks
→ documentation

Principles of Software Engineering



- **Modularity:** Divide system into independent modules.

- **Abstraction:** Focus on what a system does, not how. → interface oriented designing

- **Encapsulation:** Keep data and functions together. → classes

- **Scalability:** Software should handle growth. → vertical or horizontal (HA)

- **Reusability:** Reuse existing code to save effort. → modules

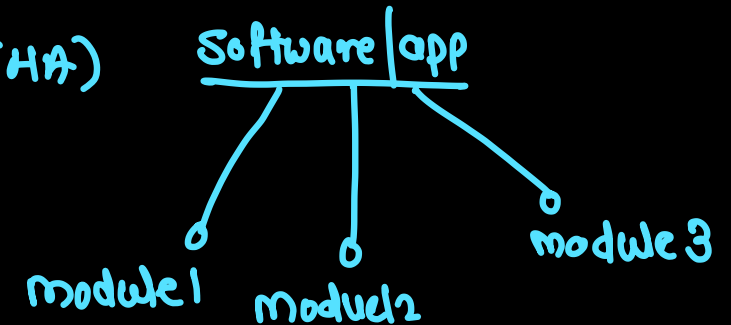
- **Maintainability:** Code should be easy to modify and fix.

- best practices

- coding guidelines

- adding comments

↪ collection of abstract methods



* E-commerce app

- user module → login, register ...
- product module → CRUD of product
- cart module → cart mgmt
- order module → order mgmt
- AI module → recommendation Engine

Characteristics of good software



Operational

development testing

↗ financial
↘ time

- Budget: Software should be developed within the budget
- Usability: Software should be usable by the end user → Simple UI, easy to understand
- Efficiently: Software should efficiently use the storage and space → resources → CPU | memory | storage
- Correctness: Software should provide all the functionalities correctly without having any bug/issue → testing
- Functionality: Software should meet all the requirements of the user
- Dependability: Software should contain all the dependencies in its package
- Security: Software should keep the data safe from any external threat → confidentiality (encryption), integrity, availability
- Safety: Software should not be hazardous or harmful to the environment
authentication, authorization

Transitional

- Portability: If the software can perform the same operations on different environments and platforms, that shows its Portability
- Interoperability: It is the ability of the software to use the information transparently
- Reusability: If on doing slight modifications to the code of the software, we can use it for a different purpose, then it is reusable → classes, modules
- Adaptability: It is an ability to adapt the new changes in the environment → resources addition/removal

Maintenance

↗ Fixing bugs & adding new features

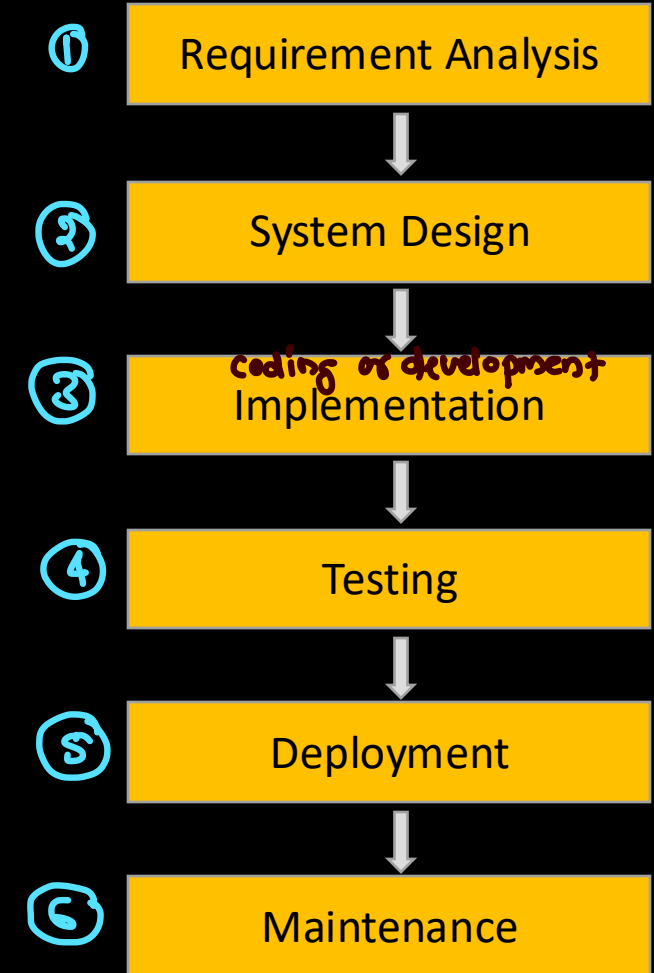
- Maintainability: The software should be easy to maintain by any user
- Flexibility: The software should be flexible to any changes made to it
- Extensibility: There should not be any problem with the software on increasing the number of functions performed by it → Extensions | plugins
- Testability: It should be easy to test the software → TDD → Test Driven Development
- Modularity: A software is of high modularity if it can be divided into separate independent parts and can be modified and tested separately



Software Development **Life Cycle**

Overview

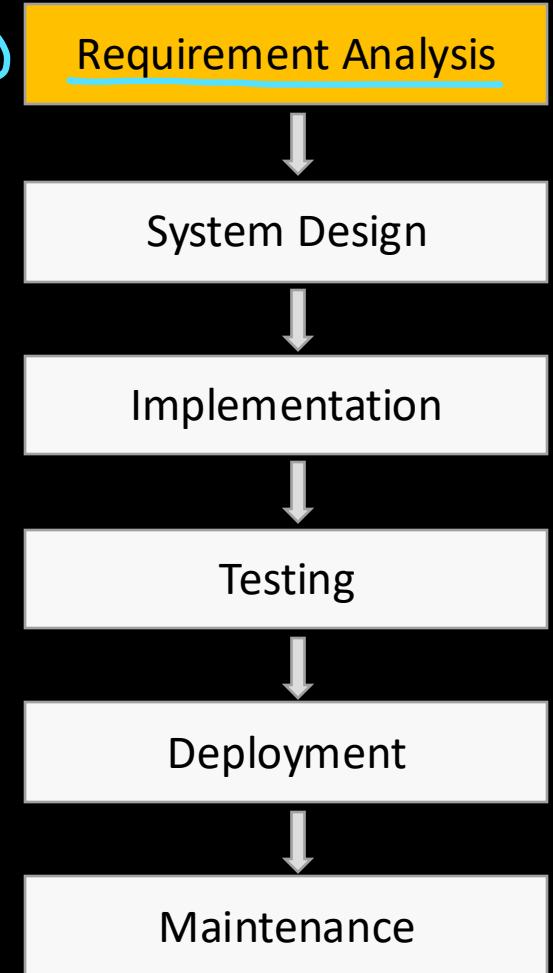
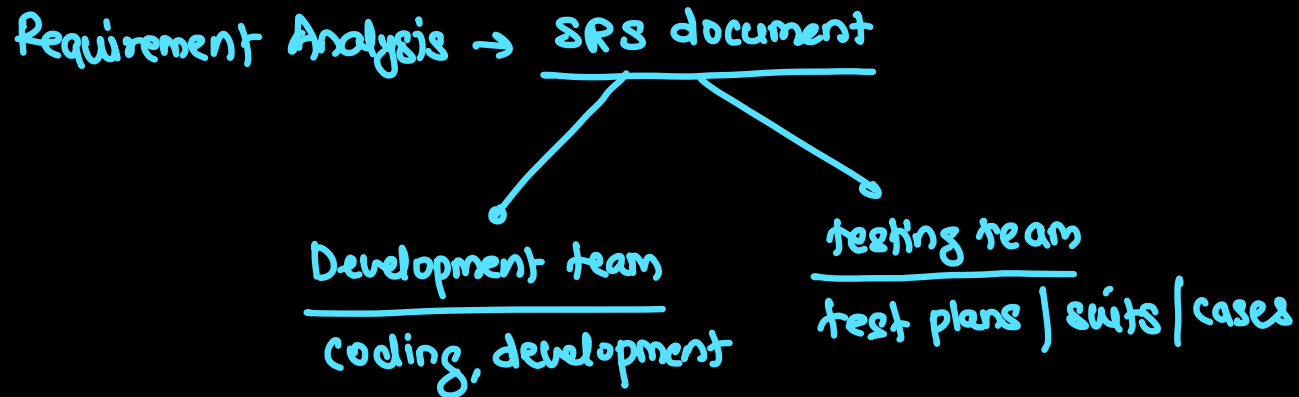
- Also called as Software Development Process (SDP)
- Is a well-defined, structured sequence of stages in software engineering to develop the intended software product
- Is a framework defining tasks performed at each step in the software development process
- Aims to produce a high-quality software that
 - meets or exceeds customer expectations
 - reaches completion within times and cost estimates → budget
- Consists of detailed plan (stages) of describing how to develop, test, deploy and maintain the software product





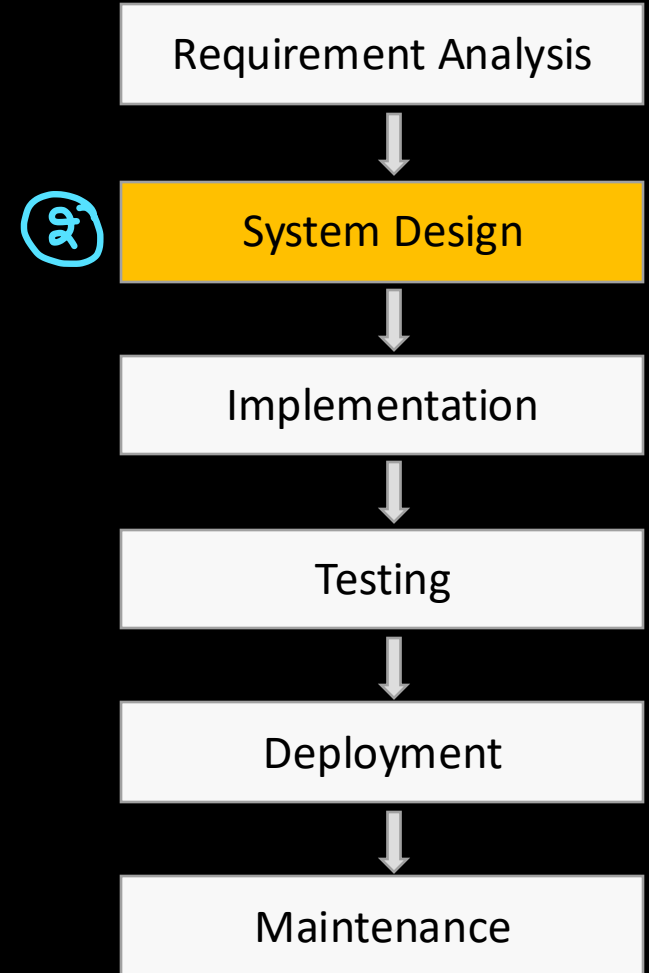
Requirement Analysis

- Goal: Understand what needs to be built
- Activities:
 - having interest in the app: investors, shareholders,
 - Gather requirements from stakeholders, clients, or end users
 - Create a Software Requirement Specification (SRS) document
 - Identify technical and business feasibility
- Output: SRS document



System Design

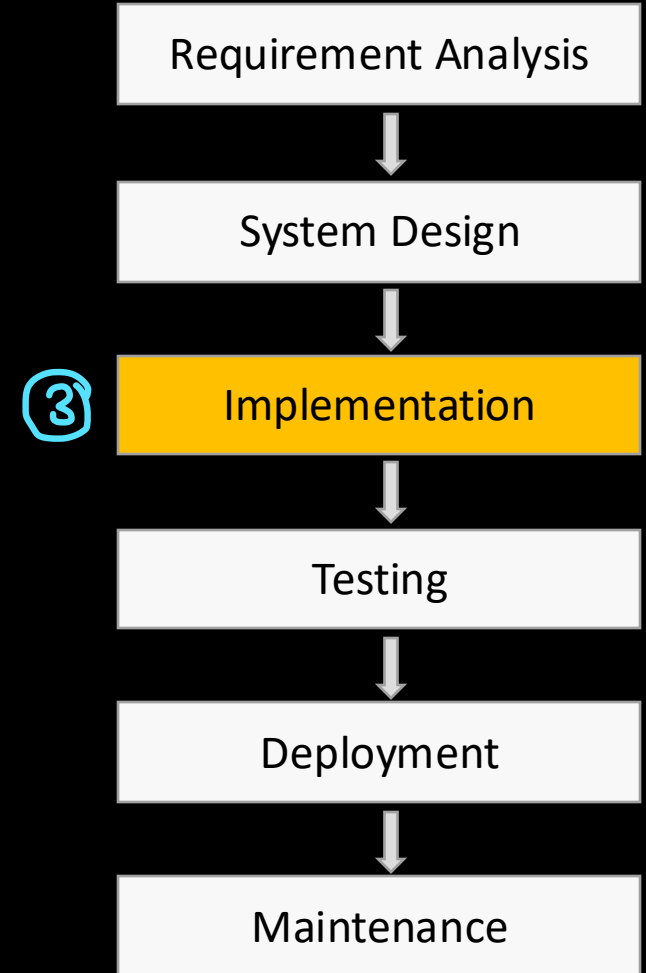
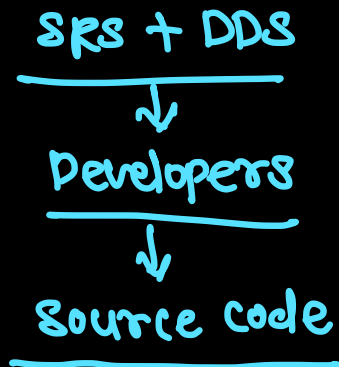
- **Goal:** Plan how the software will be built
 - **Activities:** → monolithic | microservices | n-tier architecture
 - Define architecture, data flow, and database schema → RDBMS | NoSQL
 - Create UI/UX wireframes → screen designs
 - Select technologies (frontend, backend, database, etc.)
 - **Output:** Design Documents (HLD, LLD – High/Low-Level Design)
 - technology stack
 - LAMP | WAMP | MAMP
 - MERN
 - MEAN
 - WISA
 - language
- DDS → Design Document Specification





Implementation → coding or development

- **Goal:** Convert design into working code
- **Activities:**
 - Developers write code following coding standards
 - Use version control (Git) for collaboration
 - Conduct unit testing for each module
- **Output:** Working source code



Testing

- **Goal:** Ensure software works correctly and meets requirements (SRS)
- **Activities:** → functional testing
 - Perform unit, integration, system, and user acceptance testing (UAT)
 - Identify and fix bugs → bug tracking app → Bugzilla, trello
 - Validate performance, security, and compatibility → non functional testing
- **Output:** Tested and verified application

testing types

- manual testing
- automated testing

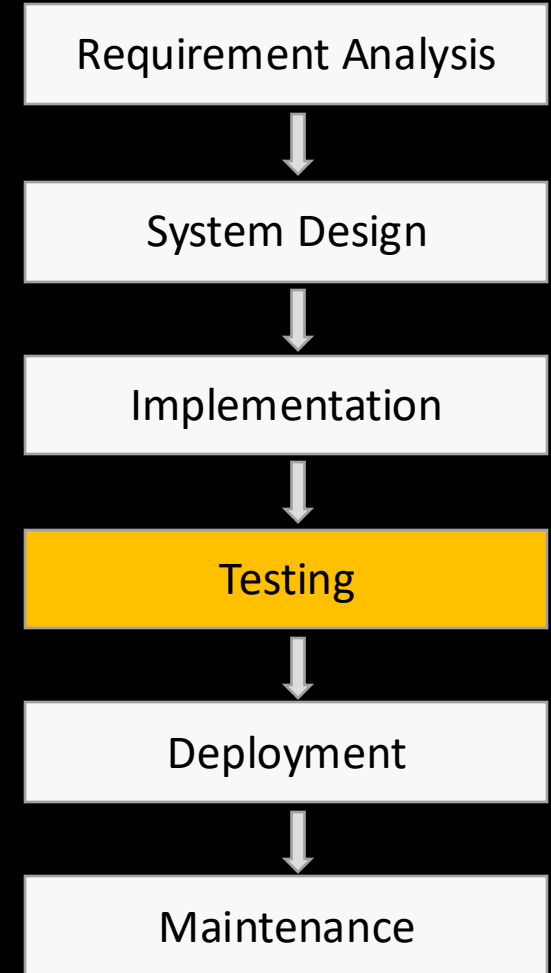
testing methods

- black box testing
- white box testing
- gray box testing

testing levels

- functional level
- non-functional level

④

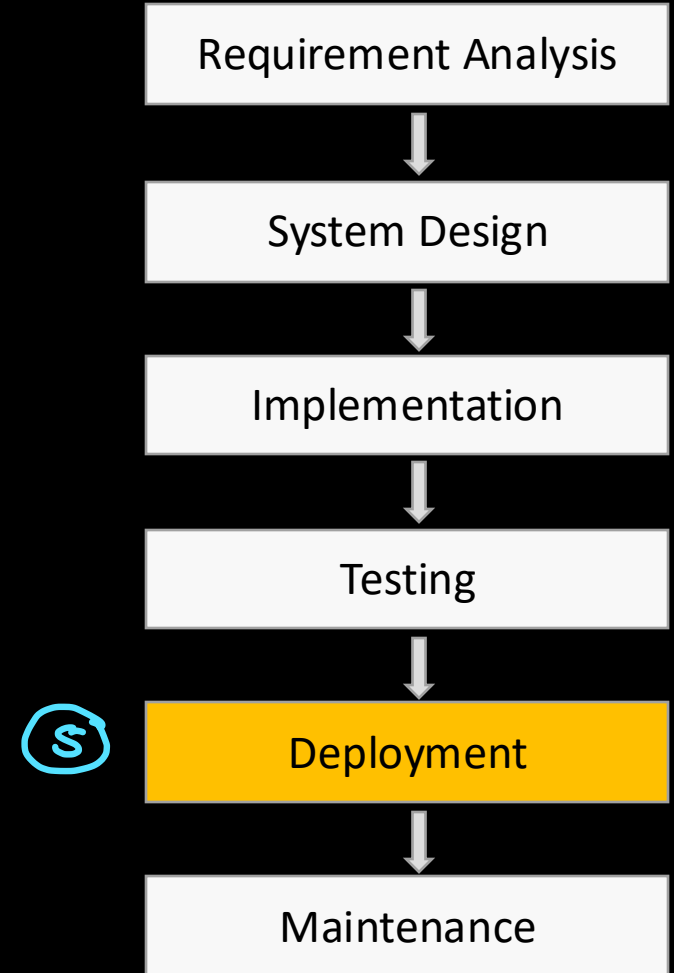


Deployment (production deployment)

- **Goal:** Deliver the product to the end users or production environment
- **Activities:**
 - Deploy software using CI/CD pipelines or manual methods
 - Conduct smoke testing in production → Quick testing
 - Monitor for early issues
- **Output:** Software available to users

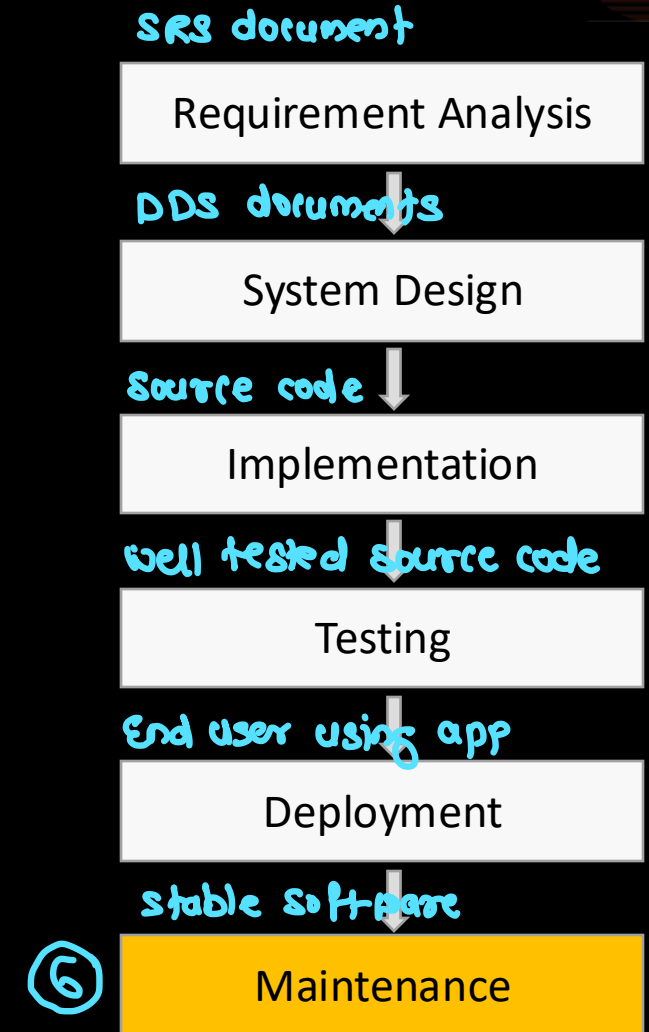
Deployment → moving an app from one to another environment

- staging deployment
- testing deployment
- pre-prod deployment
- prod deployment



Maintenance

- **Goal:** Keep the software running smoothly after release
- **Activities:**
 - Fix bugs reported by users
 - Update software with new features or security patches
 - Improve performance
- **Output:** Updated and stable software



SDLC Models



Model	Description	When to Use
<u>Waterfall</u> ^{deprecated}	<u>Linear and sequential</u> — <u>each phase follows the previous one.</u>	<u>Small, well-defined projects.</u>
✓ <u>Agile</u> ^{敏捷}	<u>Iterative and incremental</u> — <u>work in short sprints with continuous feedback.</u>	<u>Dynamic projects with evolving requirements.</u>
<u>Spiral</u>	Combines design and prototyping with risk analysis.	Large, complex, and high-risk projects.
<u>V-Model</u>	Extension of Waterfall with testing after every phase.	When strict validation is required.
<u>Iterative</u>	Develop a version, test, improve, and repeat.	When gradual improvement is needed.
✓ <u>DevOps</u> ^{持续交付}	<u>Continuous integration, delivery, and monitoring.</u>	<u>For modern, cloud-based environments needing fast releases.</u>



Benefits of SDLC

- Ensures structured development and predictability
- Improves project planning and resource management
- Reduces project risks and development costs
- Delivers high-quality software that meets requirements (SR2)



Agile Methodologies

Agile Development

can be adapted by multiple industries → construction | software development
multiple times (sprints)

- Agile Development is a flexible, iterative approach to software development that focuses on continuous delivery, collaboration, and customer feedback
- Instead of building the entire product in one go (like in the Waterfall model), Agile breaks the project into small parts called iterations or sprints – each delivering a working piece of software
- Agile = “Build small, deliver fast, get feedback, and improve continuously.”
- The Agile Manifesto (Core Philosophy)
 - In 2001, 17 software developers wrote the Agile Manifesto, which defines 4 key values and 12 principles for better software development

Traditional Focus	Agile Focus
<u>Processes and tools</u>	<u>Individuals and interactions</u>
<u>Comprehensive documentation</u>	<u>Working software</u>
<u>Contract negotiation</u>	<u>Customer collaboration</u>
<u>Following a plan</u>	<u>Responding to change</u> **

→ Events

* E-commerce app

- user module → login, register ...
- product module → CRUD of product
- cart module → cart mgmt
- order module → orders mgmt
- AI module → recommendation Engine

tasks / stories (product backlog)

- 1: create a table named users
- 2: create a backend API for registration
- 3: create a screen for user registration
- 4: integrate API in registration screen
- 5: create a backend API for login
- 6: create a screen for user login
- 7: integrate API in login screen
- 8: ...
- 9: ...
- 10: ...

→ Sprint 1

→ Sprint backlog

→ Sprint 2

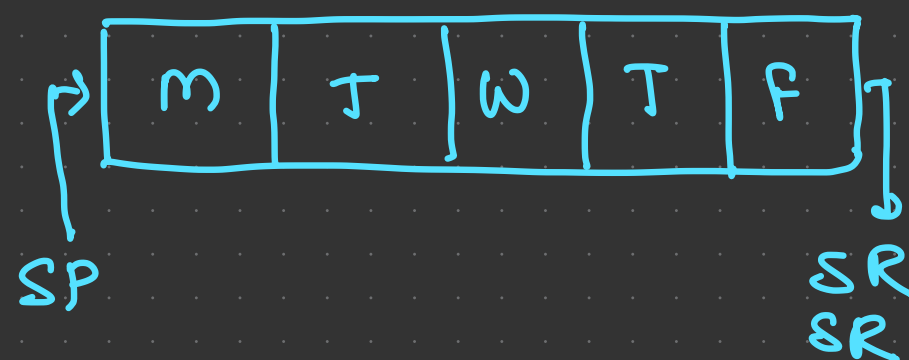
→ Sprint backlog

Sprint → time boxed event (1-4 weeks) = 1 week

Sprint 1



Sprint 2



Sprint 3



Components of Agile Development



Component	Description
<u>Sprint</u>	A short, time-boxed period (1–4 weeks) where a small part of the product is developed.
<u>Product Backlog</u>	A prioritized list of features or user stories that need to be implemented.
<u>Sprint Backlog</u>	Subset of the product backlog selected for a specific sprint.
<u>Daily Stand-up</u>	A short 15-minute daily meeting to track progress and obstacles.
<u>Sprint Review</u>	Meeting at the end of the sprint to demonstrate completed work.
<u>Sprint Retrospective</u>	A reflection session to improve team processes in the next sprint.

Sprint planning

Agile Frameworks



Framework	Highlights
<u>Scrum</u> 🧑🧑🧑	<u>Most popular Agile framework; uses sprints, roles, and ceremonies.</u>
<u>Kanban</u>	<u>Visual workflow (board with “To Do → Doing → Done”) focusing on continuous flow.</u>
<u>Extreme Programming (XP)</u>	Emphasizes coding practices like pair programming and continuous integration.
<u>Lean</u>	Focuses on minimizing waste and maximizing value delivery.
<u>SAFe (Scaled Agile Framework)</u>	Extends Agile for large, enterprise-level teams.

What is Scrum ?



- Scrum is an Agile framework used to develop, deliver, and sustain complex software projects
- It emphasizes team collaboration, iterative progress, continuous feedback, and flexibility
- Scrum helps teams deliver working software quickly, in small increments, while continuously improving
- Scrum divides the entire project into small, manageable chunks called Sprints (typically 1–4 weeks)
- Each sprint results in a potentially shippable product increment — something working and usable
- At the end of every sprint:
 - The team reviews progress
 - The product is demonstrated
 - Feedback is gathered
 - Improvements are planned for the next sprint

} Sprint review

→ Sprint Retro

Scrum Roles



■ Product Owner

- Represents the customer or stakeholders
- Manages the Product Backlog
- Prioritizes features and defines User Stories
- Ensures the team builds what provides the most value

■ Scrum Master

- Acts as a coach and facilitator for the Scrum team
- Removes obstacles (impediments) → Events
- Ensures Scrum principles and ceremonies are followed
- Shields team from distractions

■ Development Team → Scrum team

- Cross-functional members (developers, testers, designers, etc.)
- Self-organizing — decide how to build the product
- Delivers a working product increment every sprint



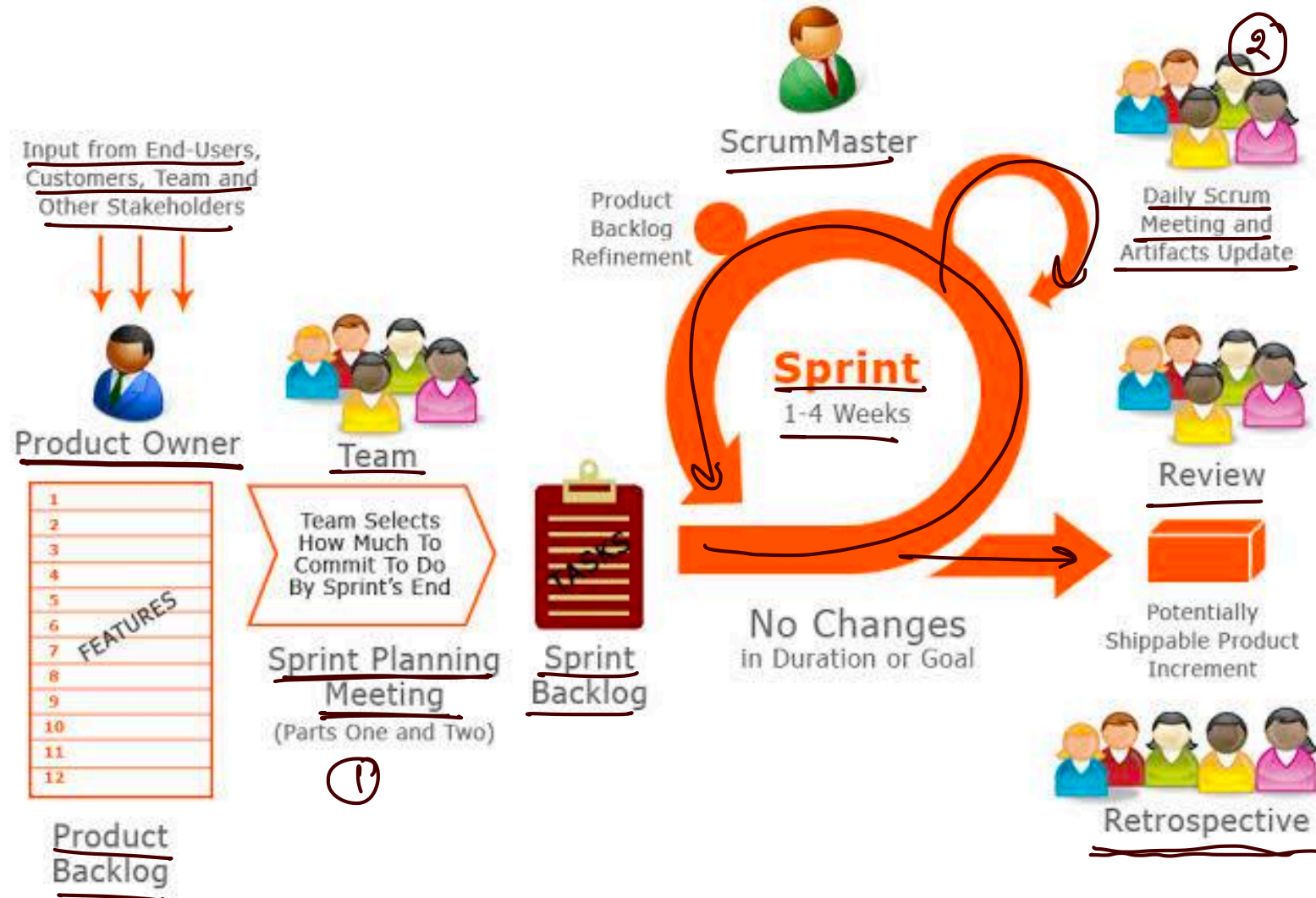
Scrum Artifacts (Key Documents)

- Product Backlog
 - A prioritized list of everything needed in the product (features, bugs, improvements)
 - Managed by the Product Owner
- Sprint Backlog
 - A subset of the Product Backlog selected for the current sprint
 - Includes tasks and a goal
- Increment
 - The working product that results from a sprint — it must be usable and meet the “Definition of Done”
- Definition of Done (DoD)
 - A shared understanding of what “done” means (e.g., coded, tested, documented, deployed)

Scrum Ceremonies (Events)



Ceremony	Purpose	Duration
<u>Sprint Planning</u>	Define the Sprint Goal and decide which backlog items to work on.	2–4 hours (for 2-week sprint)
<u>Daily Scrum (Stand-up)</u>	15-min meeting where team members share: 1. What they did yesterday 2. What they'll do today 3. Any blockers	15 minutes daily
<u>Sprint Review</u>	Demonstrate the completed work to stakeholders and collect feedback.	1–2 hours
<u>Sprint Retrospective</u>	Discuss what went well , what didn't , and how to improve next sprint.	1–1.5 hours
<u>The Sprint</u>	The actual 2–4 week development cycle where work happens.	Fixed timebox



Scrum tools



- **Jira** – <https://www.atlassian.com/software/jira/>
- **Clarizen** – <https://www.clarizen.com/>
- **GitScrum** – <https://site.gitscrum.com/>
- **Vivify Scrum** – <https://www.vivifyscrum.com/>
- **Yodiz** – <https://www.yodiz.com/>
- **ScrumDo** – <https://www.scrumdo.com/>
- **Quickscrum** – <https://www.quickscrum.com/>
- **Manuscript** – <https://www.manuscript.com/>
- **Scrumwise** – <https://www.scrumwise.com/>
- **Axosoft** – <https://www.axosoft.com/>





SRS

Application Testing

<u>Verification</u>	<u>and Validation</u>
<u>Static process</u>	<u>dynamic process</u>
<u>developers</u>	<u>testers</u>

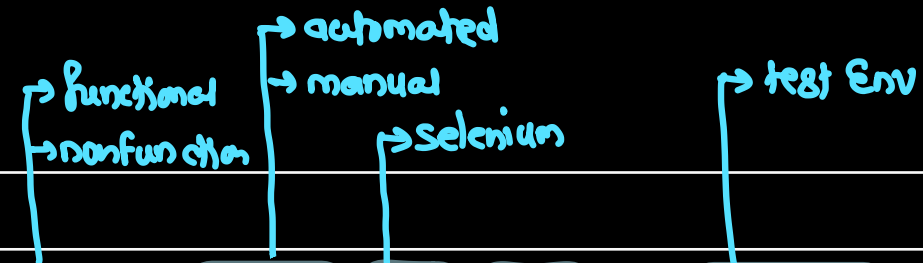
Introduction



- **Application Testing** is the process of verifying and validating that a software application:
 - Works as intended
 - Meets business and user requirements → SRS
 - Is free from defects or bugs
 - Performs well under various conditions
- In simple terms:
 - Application Testing ensures the software is reliable, secure, and user-friendly before it goes live
- **Goals of Application Testing**
 - Detect and fix bugs early
 - Ensure the functionality meets requirements (SRS)
 - Verify performance, usability, and security → non-functional testing
 - Prevent future defects through regression tests → a bug fix should not open a new bug
 - Ensure cross-platform and cross-browser compatibility (for web & mobile apps)

Phases of Application Testing → STLC

Phase	Description
<u>Test Planning</u> →	Define test objectives, strategy, tools, scope, and resources.
<u>Test Design</u> →	Create test cases, test data, and define expected outcomes.
<u>Test Environment Setup</u> →	Prepare hardware, software, and network needed for testing.
<u>Test Execution</u> →	Run test cases and record results.
<u>Defect Reporting & Tracking</u> →	Log bugs in tools (like Jira, Bugzilla).
<u>Retesting & Regression Testing</u> →	Re-test after bug fixes and ensure no new issues appear.
<u>Test Closure</u> →	Prepare summary report and finalize testing.



① login test case

- enter valid email and password
- Expected: user successfully logs in
- observed: —
- result:

② validate email

- Enter invalid email (abc)
- Expected: message shown "invalid email"
- observed: application accepted email
- result: failed

Application Testing in SDLC



SDLC Phase	Testing Involvement
<u>Requirement</u>	<u>Review for testability.</u>
<u>Design</u>	<u>Verify architecture and data flow.</u>
<u>Implementation</u>	<u>Unit & integration testing.</u>
<u>Testing</u>	<u>Functional, non-functional testing.</u>
<u>Deployment</u>	<u>Smoke testing in production.</u>
<u>Maintenance</u>	<u>Regression & patch testing.</u>

Validation →

unit = function
↳ developer



Types of Application Testing

■ Manual Testing

- Testers execute test cases without automation tools
- Focus on exploratory, usability, and ad-hoc testing
- Suitable for UI validation and small projects
- Example tools: Excel sheets, TestRail for documentation

■ Automated Testing

- Test cases are executed using automation tools or scripts
- Great for repetitive, regression, and performance tests
- Saves time and improves accuracy
- Example tools: Selenium, Cypress, Appium, JUnit, PyTest

Levels of Application Testing



Level	Purpose	Performed By	Example Tools
<u>Unit Testing</u>	<u>Test individual modules or functions.</u>	<u>Developers</u>	<u>JUnit, NUnit, PyTest</u>
<u>Integration Testing</u>	<u>Verify how modules interact together.</u>	<u>Developers/Testers</u>	<u>Postman, JUnit</u>
<u>System Testing</u>	<u>Test the complete application as a whole.</u>	<u>QA Team</u>	<u>Selenium, TestNG</u>
<u>User Acceptance Testing (UAT)</u>	<u>Validate the system with real-world scenarios.</u>	<u>End Users / Clients</u>	<u>Manual / UAT tools</u>

Types of Testing: Functional



Type	Description
<u>Smoke Testing</u>	<u>Quick test to ensure basic functions work (build verification).</u>
<u>Sanity Testing</u>	<u>Check new functionality or bug fixes.</u>
<u>Regression Testing</u>	<u>Ensure new changes don't break existing features.</u>
<u>Interface Testing</u>	<u>Validate communication between systems or APIs.</u>
<u>System Testing</u>	<u>End-to-end testing of the entire system.</u>
<u>User Acceptance Testing (UAT)</u>	Done by end-users to validate business requirements.

→ contract between service provider & service consumer

Types of Testing: Non-Functional



Type	Description
<u>Performance Testing</u>	Measure speed, scalability, and stability. → JMeter
<u>Load Testing</u>	Check system performance under expected load. → normal & peak
<u>Stress Testing</u>	Push beyond limits to see where it breaks.
<u>Security Testing</u>	Ensure data protection and access control.
<u>Usability Testing</u>	Verify ease of use and user experience.
<u>Compatibility Testing</u>	Ensure app works on different OS, browsers, and devices.
<u>Accessibility Testing</u>	Ensure app is usable by people with disabilities.

Testing Methods



Glass testing

Testing Method	Tester's Knowledge	Focus Area	Performed By	SDLC Phase
<u>White Box Testing</u>	<u>Full knowledge of internal code, logic, and structure</u>	<u>Internal workings (logic, code, paths)</u> <u>algorithms</u>	<u>Developers</u>	Implementation/ Unit testing
<u>Black Box Testing</u>	<u>No knowledge of internal code</u>	<u>Functionality and output</u>	<u>Testers / QA</u>	<u>Testing/ UAT</u>
<u>Gray Box Testing</u>	<u>Partial knowledge of internal structure</u>	<u>Combination of internal and external</u>	<u>Developers & Testers</u>	<u>Testing/Integration</u>

Common Tools for Application Testing



Category	Popular Tools
<u>Functional Testing</u>	<u>Selenium</u> , <u>Cypress</u> , <u>Playwright</u> , <u>TestNG</u>
<u>API Testing</u>	<u>Postman</u> , <u>SoapUI</u> , <u>Rest Assured</u> , <u>Thunder Client</u> , <u>Insomnia</u>
<u>Performance Testing</u>	<u>JMeter</u> , <u>Gatling</u> , <u>LoadRunner</u>
<u>Mobile Testing</u>	<u>Appium</u> , <u>Espresso</u> , <u>XCUITest</u>
<u>Security Testing</u>	<u>OWASP ZAP</u> , <u>Burp Suite</u>
<u>Test Management</u>	<u>Jira</u> , <u>TestRail</u> , <u>Zephyr</u>
<u>CI/CD Integration</u>	<u>Jenkins</u> , <u>GitHub Actions</u>



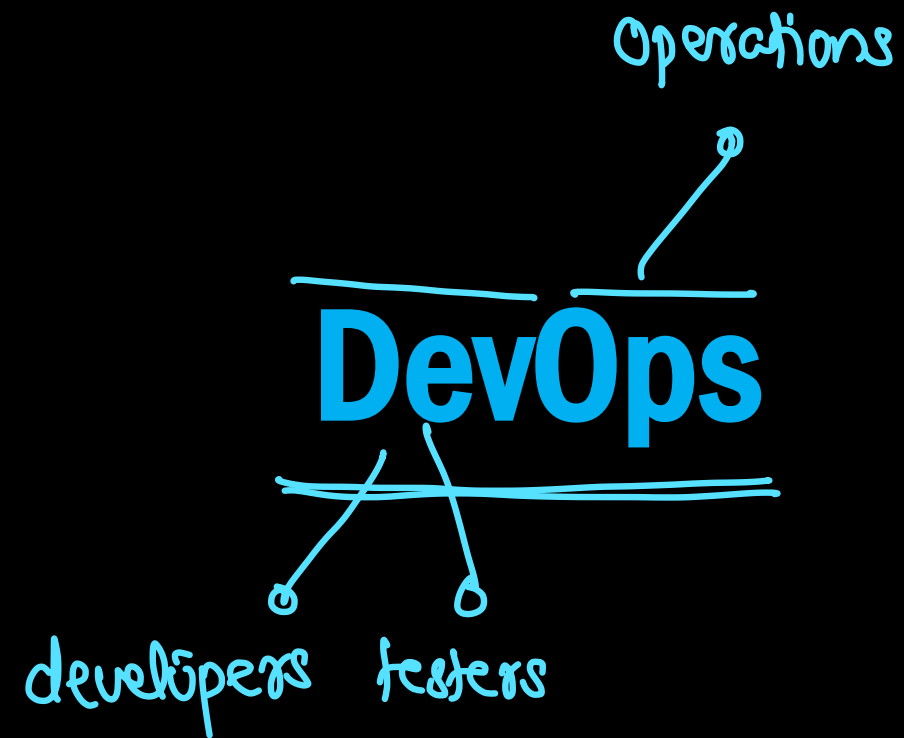
Benefits of Application Testing

- Ensures software quality and reliability
- Detects defects early, reducing costs
- Enhances user satisfaction
- Improves security and performance
- Supports continuous delivery (CI/CD) pipelines



Challenges in Application Testing

- Handling frequent code changes in Agile
- Testing across many browsers/devices
- Managing complex test data
- Balancing speed vs. depth in automated tests



Responsibilities



Dev Team

• Developers

- Requirement understanding (SRS)
- Coding → language finalized in DDS
- Version control → CVS, DVCS
- Unit testing → function testing
- Documentation → using code or manual
- Collaboration → with other developers

• Testers

- Requirement analysis (SRS)
- Test planning, designing and execution
- Defect tracking
- Reporting
- collaboration



Operations Team

- Infrastructure Management → manual or Iac
- Security & Compliance → VAPT
- Deployment & Release → in different environment
- Management
- Monitoring, Logging & Alerting → monitoring & observability
- Incident Management & Troubleshooting
- Cloud & Cost Optimization
- Backup & Disaster Recovery
- Collaboration & Support
- Performance & Capacity Planning



Challenges



Dev Team



- Environment inconsistency
- Delayed feedback loop
- Integration issues
- Manual build and deployment
- Poor visibility after deployment
- Pressure for faster delivery
- Communication gaps

Operations Team



- Frequent and sometimes unstable releases
- Lack of environment standardization
- Manual configuration management
- Limited collaboration with dev team
- Poor monitoring and alert fatigue
- Downtime and ~~indent~~ pressure
- Security and compliance

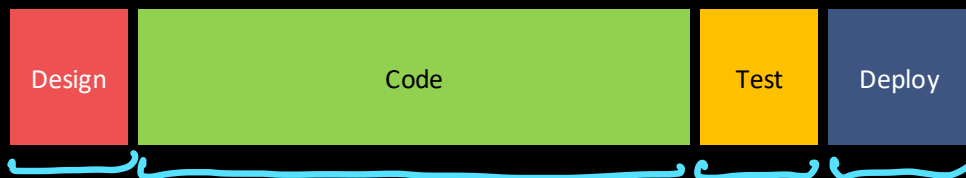
Waterfall Vs Agile



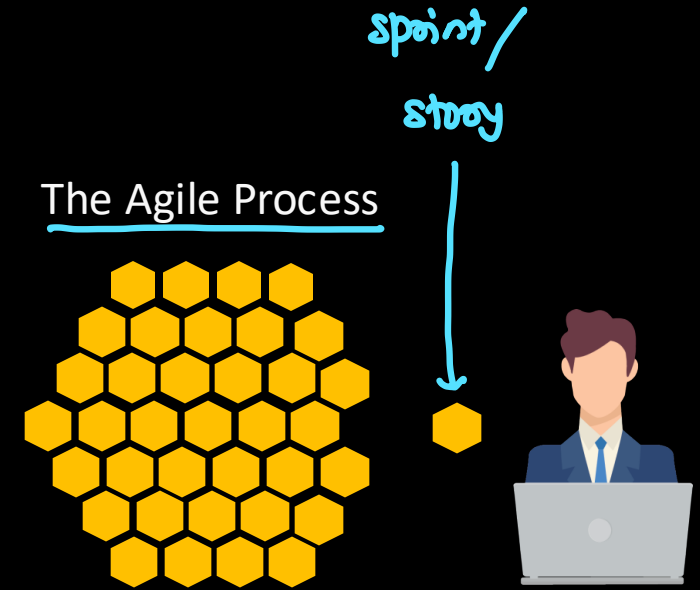
The Waterfall Process



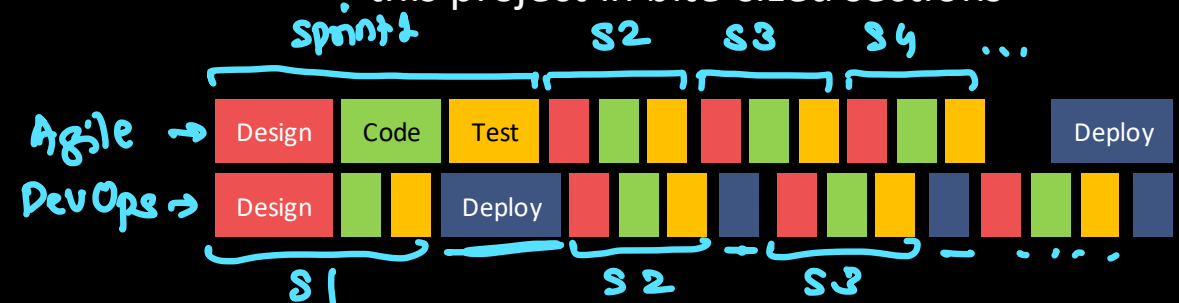
This project has got so big.
I am not sure I will be able to deliver it!



The Agile Process



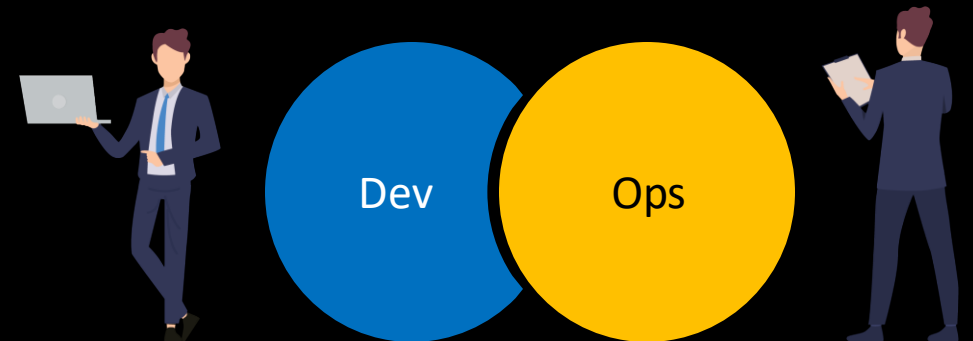
It is so much better delivering
this project in bite-sized sections



What is DevOps ?

devs testers

- DevOps is a combination of “Development” and “Operations.”
- It's both a culture and a set of practices/tools aimed at improving collaboration between software developers (Dev) and IT operations (Ops) teams
- The main goal is to deliver software faster, more reliably, and with higher quality — by automating and integrating the processes of software development and IT operations
- It's not just a tool or role, but a culture, methodology, and set of practices that bring together: Developers (who build software) and Operations (who deploy and manage it) to work collaboratively, with the goal of: Delivering software faster, more reliably, and continuously to end-users



How DevOps Solves Challenges



DevOps Solution	What It Fixes	Tools / Practice
✓ <u>Continuous Integration (CI)</u>	<u>Merge & test code frequently</u>	<u>Jenkins, GitHub Actions, GitLab CI</u>
✓ <u>Continuous Delivery (CD)</u>	<u>Automate deployment pipelines</u>	<u>ArgoCD, Spinnaker, Bamboo</u>
✓ <u>Infrastructure as Code (IaC)</u>	<u>Standardize environments</u>	<u>Terraform, Ansible, Puppet, <i>chef</i></u>
✓ <u>Containerization</u>	<u>Eliminate “works on my machine”</u>	<u>Docker, Kubernetes</u>
✓ <u>Monitoring & Logging</u>	<u>Give both teams visibility</u>	<u>Prometheus, Grafana, ELK Stack</u>
<u>Collaboration Tools</u>	<u>Improve communication</u>	<u>Slack, Microsoft Teams, Jira</u>
<u>Automation Testing</u>	<u>Faster feedback for devs</u>	<u>Selenium, Cypress, JMeter</u>

Goals of DevOps



- Faster delivery: Shorten time from idea - deployment → CI/CD
- Better collaboration: Break silos between Dev and Ops
- Automation: Reduce manual errors and repetitive work
- Reliability: Ensure systems are stable and scalable
- Continuous improvement: Use feedback to improve processes

Key Principles of DevOps



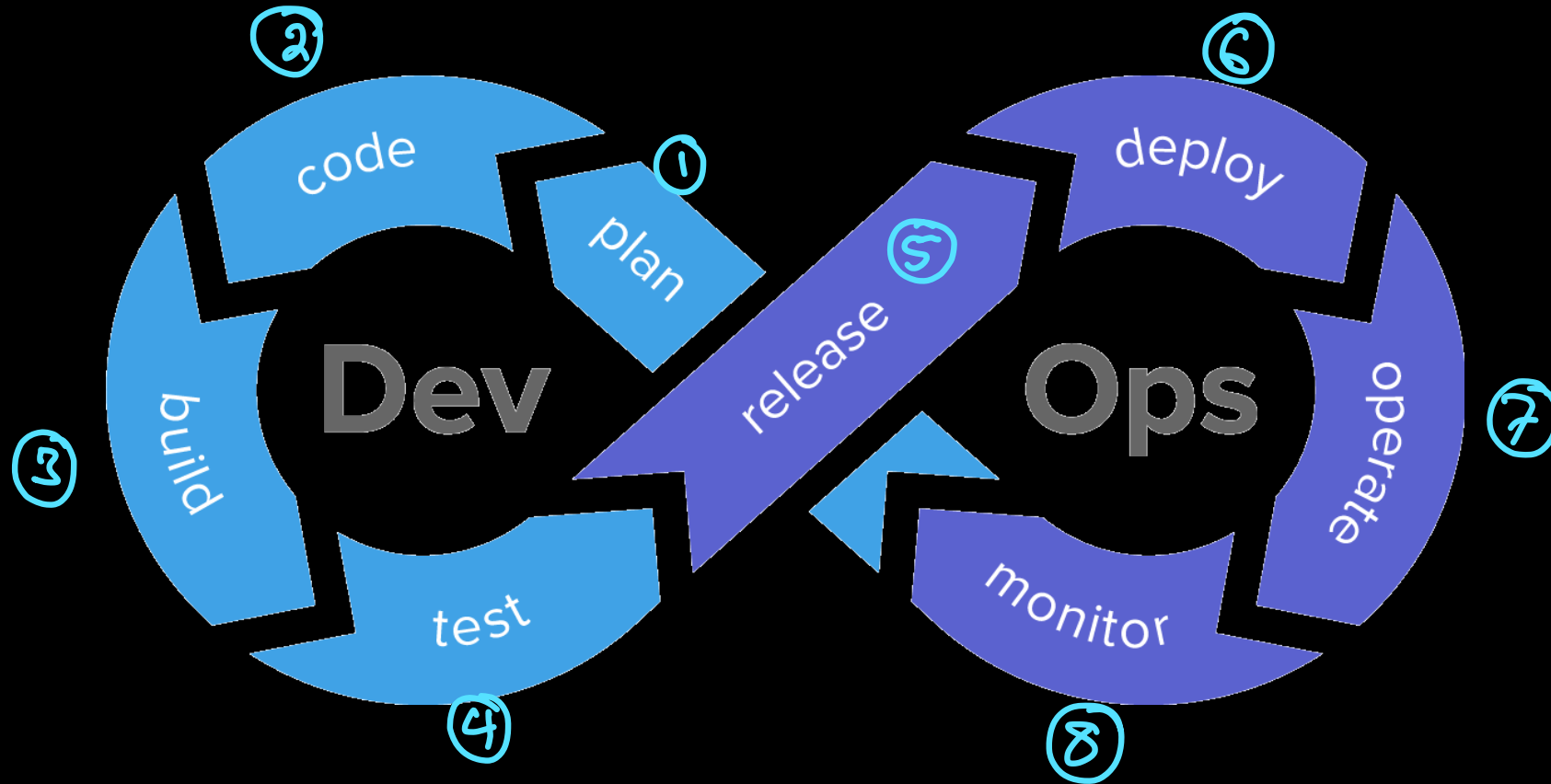
Principle	Description
Collaboration & Communication	Break silos between Dev, QA, and Ops. Everyone shares responsibility for success.
Automation	Automate everything: builds, testing, deployment, monitoring - to reduce manual errors.
Continuous Integration (CI)	Developers merge code frequently and automatically test it.
Continuous Delivery (CD)	Automatically release tested code to staging/production environments.
Continuous Monitoring	Track system health, user feedback, and performance in real-time.
Infrastructure as Code (IaC)	Manage infrastructure (servers, networks) through code, not manual setup.
Continuous Feedback	Learn from real-time data to improve the next iteration.

Reasons to use DevOps



- **Predictability**
 - DevOps offers significantly lower failure rate of new releases
- **Reproducibility**
 - Version everything so that earlier version can be restored anytime
- **Maintainability**
 - Effortless process of recovery in the event of a new release crashing or disabling the current system
- **Time to market**
 - DevOps reduces the time to market up to 50% through streamlined software delivery
 - This is particularly the case for digital and mobile applications
- **Greater Quality**
 - DevOps helps the team to provide improved quality of application development as it incorporates infrastructure issues
- **Reduced Risk**
 - DevOps incorporates security aspects in the software delivery lifecycle. It helps in reduction of defects across the lifecycle
- **Resiliency**
 - The Operational state of the software system is more stable, secure, and changes are auditable

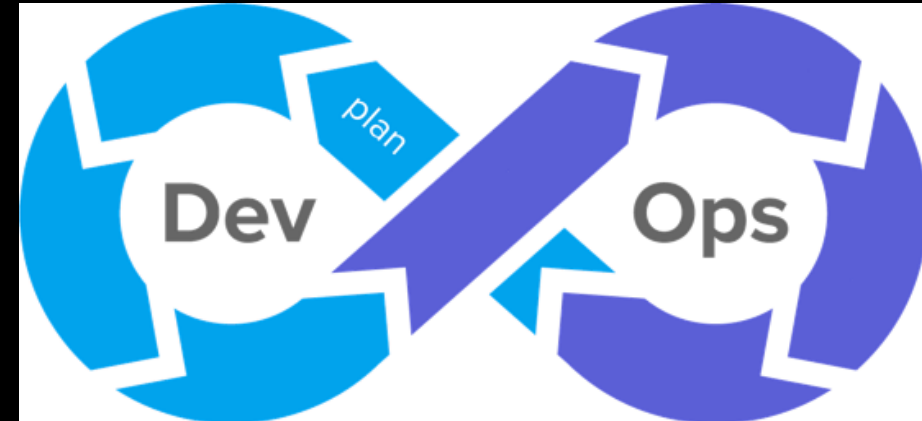
DevOps Lifecycle → Continuous → never Ending



PLAN – Planning and Tracking

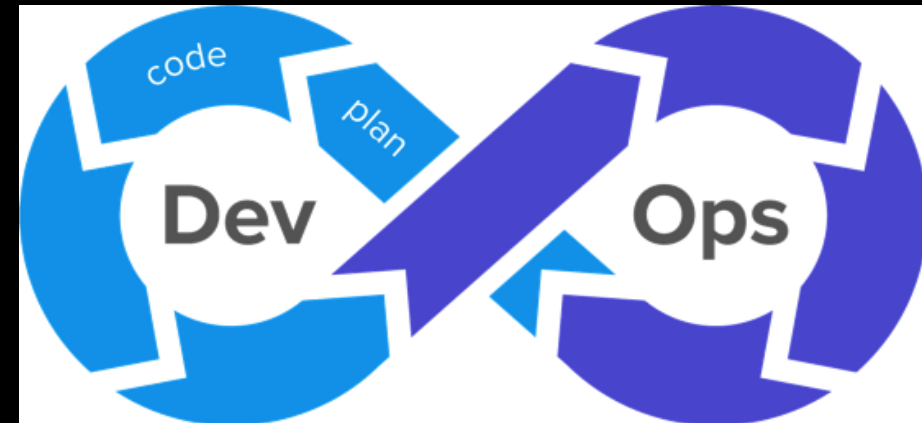


- **Goal:** Define project goals, features, and timelines (sprint)
- **Activities:**
 - Gather and prioritize requirements
 - Plan sprints, track progress, and manage tasks
 - Collaborate between development, QA, and operations
- **Common Tools:**
 - Agile Project Management: Jira, Trello, Asana, ClickUp, Azure Boards
 - Documentation: Confluence, Notion, Google Docs



CODE – Development

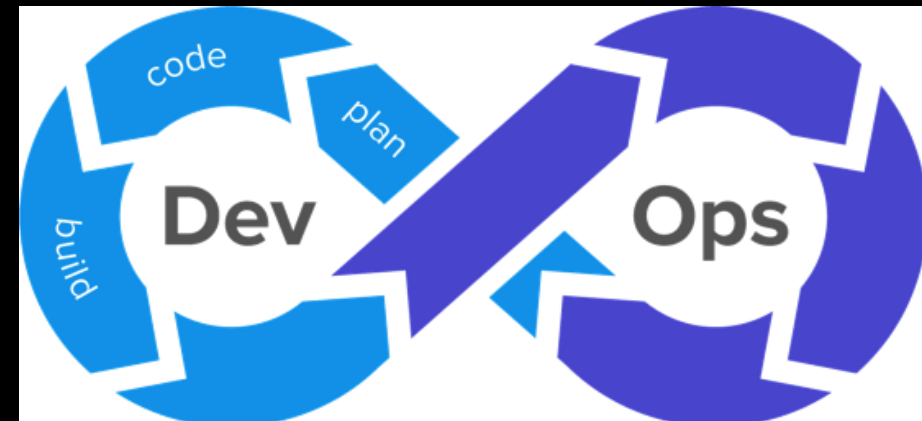
- **Goal:** Write, review, and manage code efficiently
- **Activities:**
 - Code development using IDEs
 - Version control via ~~Git~~ **VCs**
 - Code review and collaboration
- **Common Tools:**
 - Version Control: **Git**, GitHub, GitLab, Bitbucket, *CVS, SVN, bazaar*
 - Code Review: GitHub PRs, Gerrit, Crucible
 - IDEs: VS ~~Code~~, IntelliJ IDEA, PyCharm, Eclipse
 - Code Quality: SonarQube, Codacy, ESLint, Prettier
 - Editor: **VS code**, **vim**, atom, Sublime, cursor, nano





BUILD – Integration and Compilation

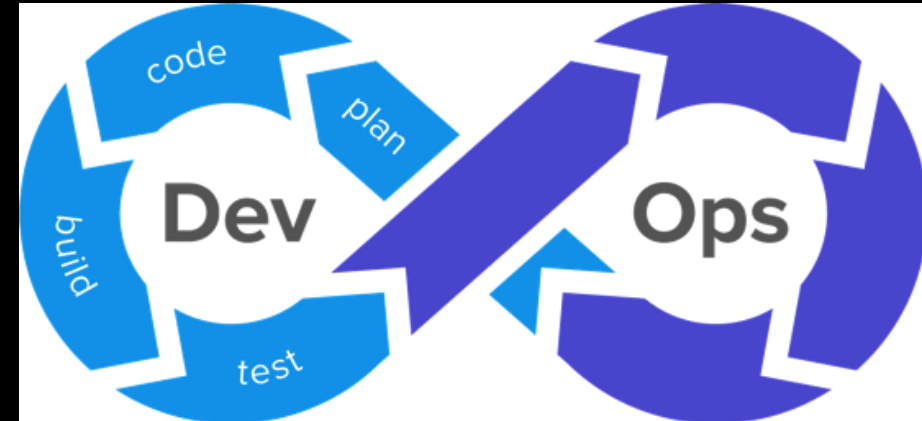
- **Goal:** Compile source code into executable artifacts → docker image, executable ...
- **Activities:**
 - Continuous Integration (CI)
 - Build automation and packaging
 - Dependency management
- **Common Tools:**
 - Build Automation: Maven, Gradle, Ant, npm, Make
 - CI Servers: Jenkins, GitLab CI, CircleCI, GitHub Actions, TeamCity
 - Artifact Repository: Nexus, JFrog Artifactory, AWS CodeArtifact



TEST – Automated Testing



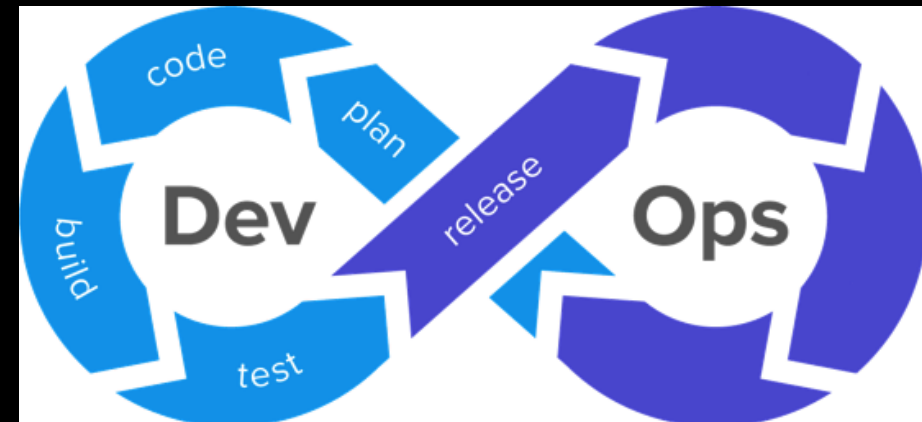
- **Goal:** Ensure software quality and reliability
- **Activities:**
 - Run automated tests (unit, integration, functional)
 - Identify bugs early in the CI pipeline
- **Common Tools:**
 - Unit Testing: JUnit, PyTest, NUnit, Mocha, Jest
 - Functional/UI Testing: Selenium, Cypress, Playwright
 - API Testing: Postman, RestAssured, SoapUI
 - Performance Testing: JMeter, Gatling, k6
 - Security Testing: OWASP ZAP, Burp Suite, SonarQube Security



RELEASE – Versioning and Approval



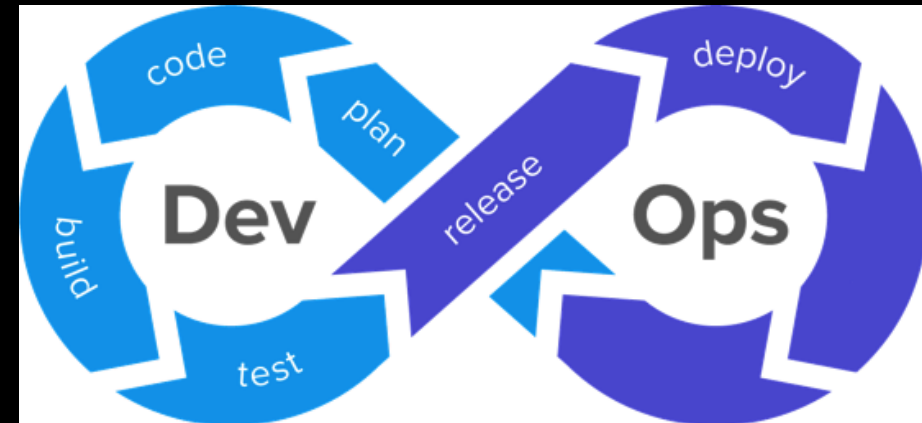
- **Goal:** Prepare the build for deployment into production-like environments
- **Activities:**
 - Version tagging, change approval, and documentation
 - Store tested builds for release
- **Common Tools:**
 - Release Automation: Jenkins, GitLab CI/CD, Bamboo, Azure DevOps, ArgoCD
 - Version Control: Git Tags, Semantic Versioning, GitHub Releases
 - Change Management: ServiceNow, Jira Service Management



DEPLOY – Continuous Deployment / Delivery



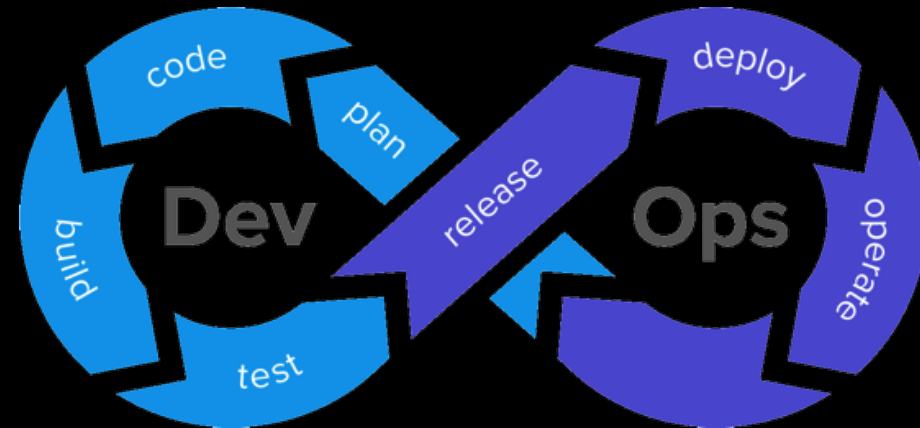
- **Goal:** Deliver the software to target environments (staging, production)
- **Activities:**
 - Continuous Delivery (manual approval)
 - Continuous Deployment (fully automated)
 - Environment configuration via Infrastructure as Code (IaC)
- **Common Tools:**
 - Containerization: Docker, Podman
 - Orchestration: Kubernetes, OpenShift, Docker Swarm
 - IaC (Infrastructure as Code): Terraform, Ansible, Puppet, Chef
 - Cloud Platforms: AWS, Azure, GCP, DigitalOcean



OPERATE – Configuration and Infrastructure Management



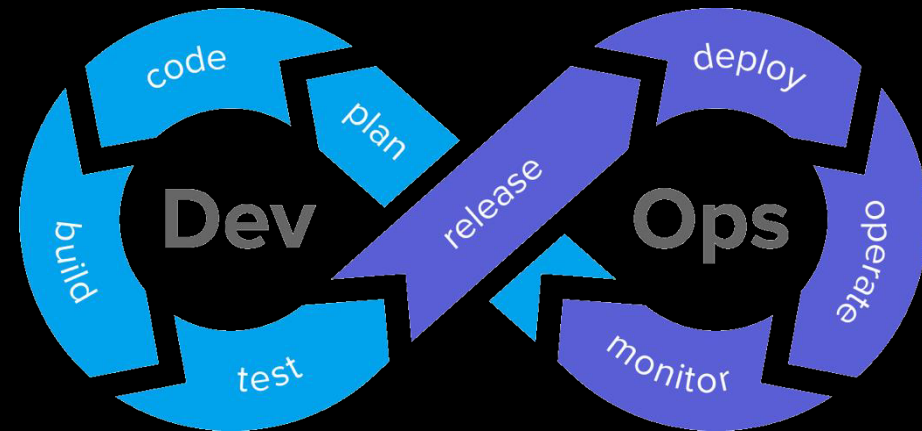
- **Goal:** Manage and maintain systems after deployment
- **Activities:**
 - Monitor uptime, performance, and incidents
 - Automate infrastructure provisioning and scaling
- **Common Tools:**
 - Configuration Management: Ansible, Puppet, Chef, SaltStack
 - Server Monitoring: Nagios, Datadog, New Relic
 - Log Management: ELK Stack (Elasticsearch, Logstash, Kibana), Fluentd, Splunk



MONITOR – Continuous Feedback



- **Goal:** Continuously observe system health, user experience, and performance
- **Activities:**
 - Collect and analyze metrics
 - Detect failures and trigger alerts
 - Feed insights back into planning phase
- **Common Tools:**
 - Monitoring: Prometheus, Grafana, Zabbix, Datadog
 - Logging: ELK Stack, Graylog, Splunk, Fluentd
 - Alerting: PagerDuty, Opsgenie, Slack Alerts
 - Feedback: Sentry, New Relic, AppDynamics



DevOps Toolchain Example



Stage	Tools
<u>Plan</u>	<u>Jira, Confluence, Notion</u>
<u>Code</u>	<u>Git, GitHub, Bitbucket</u>
<u>Build</u>	<u>Jenkins, Maven, Gradle</u>
<u>Test</u>	<u>Selenium, Cypress, JMeter</u>
<u>Release</u>	<u>GitLab CI/CD, Bamboo</u>
<u>Deploy</u>	<u>Docker, Kubernetes, ArgoCD</u>
<u>Operate</u>	<u>Ansible, Puppet, Terraform</u>
<u>Monitor</u>	<u>Prometheus, Grafana, ELK Stack</u>

Responsibilities of DevOps Engineer

